JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class FileStore

java.lang.Object
    java.nio.file.FileStore

```
public abstract class FileStore
extends Object
```

Storage for files. A `FileStore` represents a storage pool, device, partition, volume, concrete file system or other implementation specific means of file storage. The `FileStore` for where a file is stored is obtained by invoking the `getFileStore` method, or all file stores can be enumerated by invoking the `getFileStores` method.

In addition to the methods defined by this class, a file store may support one or more `FileStoreAttributeView` classes that provide a read-only or updatable view of a set of file store attributes.

Since:

1.7

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**FileStore**()	Initializes a new instance of this class.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
abstract Object	**getAttribute**(String attribute)	Reads the value of a file store attribute.
long	**getBlockSize**()	Returns the number of bytes per block in this file store.
abstract <V extends FileStoreAttributeVi V	**getFileStoreAttributeVie**(Class<V> type)	Returns a `FileStoreAttributeView` of the given type.

abstract long	**getTotalSpace**()	Returns the size, in bytes, of the file store.
abstract long	**getUnallocatedSpace**()	Returns the number of unallocated bytes in the file store.
abstract long	**getUsableSpace**()	Returns the number of bytes available to this Java virtual machine on the file store.
abstract boolean	**isReadOnly**()	Tells whether this file store is read-only.
abstract String	**name**()	Returns the name of this file store.
abstract boolean	**supportsFileAttributeVie**(Class<? extends FileAttributeView> type)	Tells whether or not this file store supports the file attributes identified by the given file attribute view.
abstract boolean	**supportsFileAttributeVie**(String name)	Tells whether or not this file store supports the file attributes identified by the given file attribute view.
abstract String	**type**()	Returns the *type* of this file store.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

FileStore

```
protected FileStore()
```

Initializes a new instance of this class.

Method Details

name

```
public abstract String name()
```

Returns the name of this file store. The format of the name is highly implementation specific. It will typically be the name of the storage pool or volume.

The string returned by this method may differ from the string returned by the `toString` method.

Returns:

the name of this file store

type

```
public abstract String type()
```

Returns the *type* of this file store. The format of the string returned by this method is highly implementation specific. It may indicate, for example, the format used or if the file store is local or remote.

Returns:

a string representing the type of this file store

isReadOnly

```
public abstract boolean isReadOnly()
```

Tells whether this file store is read-only. A file store is read-only if it does not support write operations or other changes to files. Any attempt to create a file, open an existing file for writing etc. causes an `IOException` to be thrown.

Returns:

`true` if, and only if, this file store is read-only

getTotalSpace

```
public abstract long getTotalSpace()
                            throws IOException
```

Returns the size, in bytes, of the file store. If the total number of bytes in the file store is greater than `Long.MAX_VALUE`, then `Long.MAX_VALUE` will be returned.

Returns:

the size of the file store, in bytes

Throws:

`IOException` - if an I/O error occurs

getUsableSpace

```
public abstract long getUsableSpace()
                            throws IOException
```

Returns the number of bytes available to this Java virtual machine on the file store. If the number of bytes available is greater than `Long.MAX_VALUE`, then `Long.MAX_VALUE` will be returned.

The returned number of available bytes is a hint, but not a guarantee, that it is possible to use most or any of these bytes. The number of usable bytes is most likely to be accurate immediately after this method returns. It is likely to be made inaccurate by any external I/O operations including those made on the system outside of this Java virtual machine.

Returns:

the current number of usable bytes

Throws:

`IOException` - if an I/O error occurs

getUnallocatedSpace

```
public abstract long getUnallocatedSpace()
                                  throws IOException
```

Returns the number of unallocated bytes in the file store. If the number of unallocated bytes is greater than `Long.MAX_VALUE`, then `Long.MAX_VALUE` will be returned.

The returned number of unallocated bytes is a hint, but not a guarantee, that it is possible to use most or any of these bytes. The number of unallocated bytes is most likely to be accurate immediately after this method returns. It is likely to be made inaccurate by any external I/O operations including those made on the system outside of this virtual machine.

Returns:

the current number of unallocated bytes

Throws:

`IOException` - if an I/O error occurs

getBlockSize

```
public long getBlockSize()
                  throws IOException
```

Returns the number of bytes per block in this file store.

File storage is typically organized into discrete sequences of bytes called *blocks*. A block is the smallest storage unit of a file store. Every read and write operation is performed on a multiple of blocks.

Implementation Requirements:

The implementation in this class throws `UnsupportedOperationException`.

Returns:

a positive value representing the block size of this file store, in bytes

Throws:

`IOException` - if an I/O error occurs

`UnsupportedOperationException` - if the operation is not supported

Since:

10

supportsFileAttributeView

```
public abstract boolean supportsFileAttributeView
(Class<? extends FileAttributeView> type)
```

Tells whether or not this file store supports the file attributes identified by the given file attribute view.

Invoking this method to test if the file store supports `BasicFileAttributeView` will always return `true`. In the case of the default provider, this method cannot guarantee to give the correct result when the file store is not a local storage device. The reasons for this are implementation specific and therefore unspecified.

Parameters:

`type` - the file attribute view type

Returns:

`true` if, and only if, the file attribute view is supported

supportsFileAttributeView

```
public abstract boolean supportsFileAttributeView(String name)
```

Tells whether or not this file store supports the file attributes identified by the given file attribute view.

Invoking this method to test if the file store supports `BasicFileAttributeView`, identified by the name "`basic`" will always return `true`. In the case of the default provider, this method cannot guarantee to give the correct result when the file store is not a local storage device. The reasons for this are implementation specific and therefore unspecified.

Parameters:

`name` - the `name` of file attribute view

Returns:

`true` if, and only if, the file attribute view is supported

getFileStoreAttributeView

```
public abstract <V extends FileStoreAttributeView>
V getFileStoreAttributeView(Class<V> type)
```

Returns a `FileStoreAttributeView` of the given type.

This method is intended to be used where the file store attribute view defines type-safe methods to read or update the file store attributes. The `type` parameter is the type of the attribute view required and the method returns an instance of that type if supported.

Type Parameters:

`V` - The `FileStoreAttributeView` type

Parameters:

`type` - the `Class` object corresponding to the attribute view

Returns:

a file store attribute view of the specified type or `null` if the attribute view is not available

getAttribute

```
public abstract Object getAttribute(String attribute)
                                throws IOException
```

Reads the value of a file store attribute.

The `attribute` parameter identifies the attribute to be read and takes the form:

view-name:*attribute-name*

where the character `':'` stands for itself.

view-name is the `name` of a `AttributeView` that identifies a set of file attributes. *attribute-name* is the name of the attribute.

Usage Example: Suppose we want to know if ZFS compression is enabled (assuming the "zfs" view is supported):

```
    boolean compression = (Boolean)fs.getAttribute("zfs:compression");
```

Parameters:

`attribute` - the attribute to read

Returns:

the attribute value; `null` may be valid for some attributes

Throws:

`UnsupportedOperationException` - if the attribute view is not available or it does not support reading the attribute

`IOException` - if an I/O error occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.